

Dask on HTCondor for HEP

Jasin Mustafi, 15.07.2026

Documentation [Link](#)

Dask on HTCondor for HEP

HiWi job goal:

- Proof-of-concept Dask cluster for HEP analyses
- JupyterHub and Jupyter notebooks for interactive analyses
- Dask gateway for automatic deployment on ETP HTCondor batch system
- JupyterHub authentication via LDAP
- Hosted on bms1.etp.kit.edu

Outline

- JupyterHub
- Dask
- Implementation
- Demonstration
- Outlook

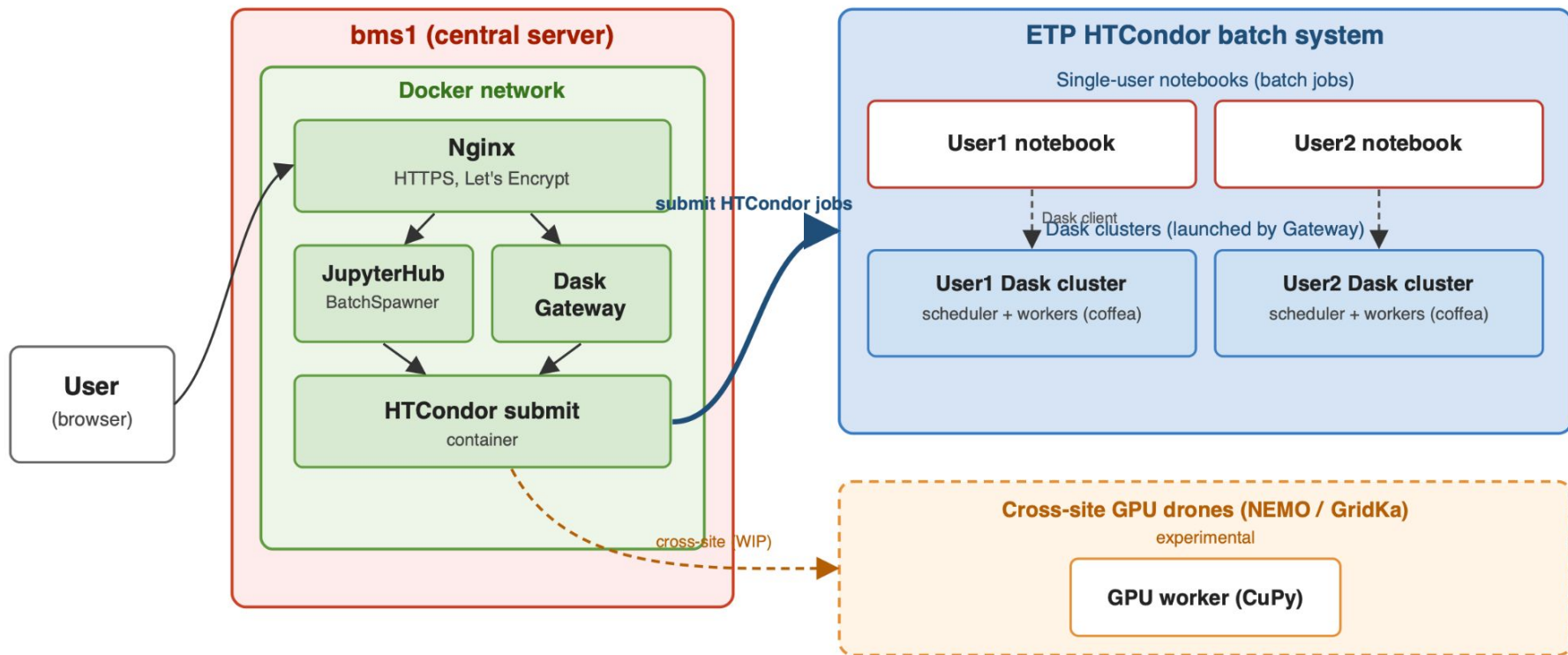
JupyterHub

- Centralized server to manage and spawn isolated user environments
- User access through browser
- BatchSpawner (HTCondor)
 - LDAP authentication
 - Create notebook from Docker image
 - Mount user's home, work, and ceph directory

Dask

- Python library for parallel computing: lazy evaluation, scheduling
- Dask cluster:
 - Client: Interface to submit to scheduler (user notebook)
 - Scheduler: Coordination of workers
 - Workers: Coffea (awkward arrays + uproot)
- Dask gateway:
 - Manage dask clusters
 - Uses HTCondor (distributed batch system for resource isolation)

Implementation (on bms1)



Implementation

- Docker Compose stack on bms1
- HTCondor submit container submits all jobs to the ETP batch system
- Services in an internal Docker network:
 - JupyterHub (BatchSpawner) spawns notebooks as HTCondor jobs
 - Dask Gateway launches Dask scheduler + workers as HTCondor jobs
 - Nginx: HTTPS (Let's Encrypt) + routing to Hub and dashboard
 - Security: LDAP login, Jupyter token for Gateway auth

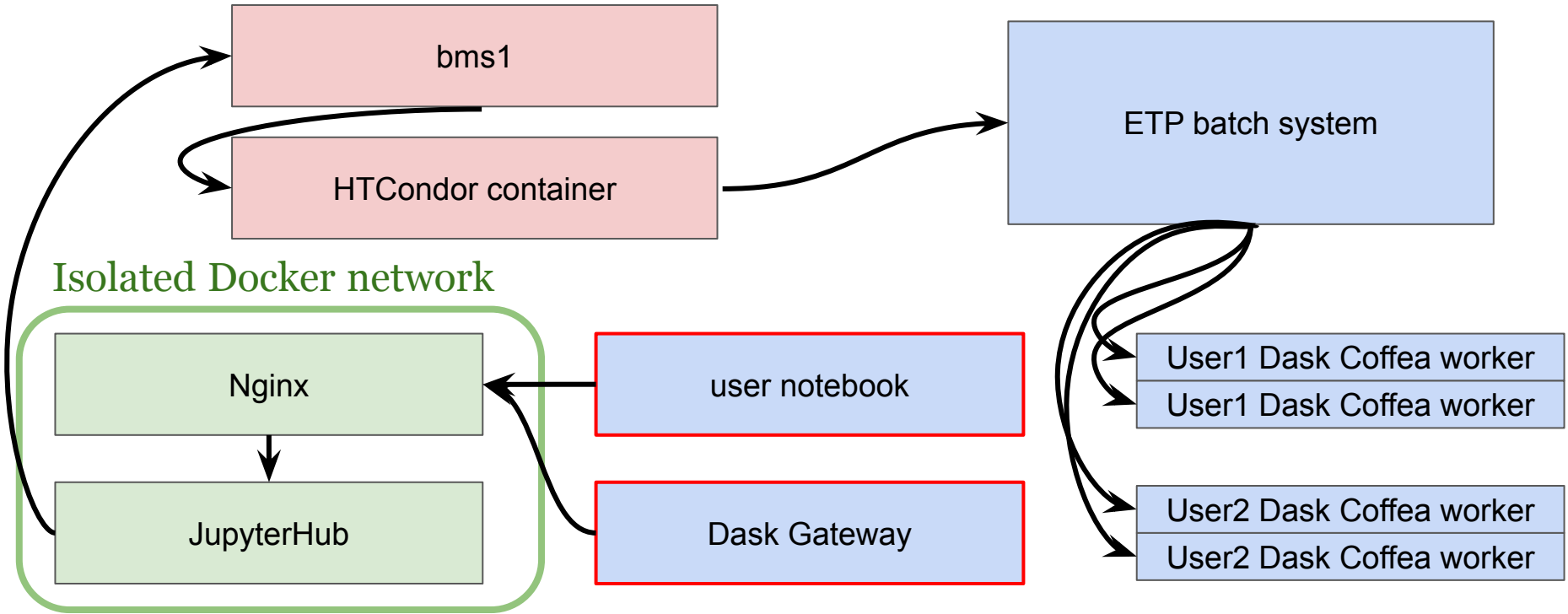
Live Demonstration

Outlook

- Next steps
 - Framework validated with a real HEP analysis (coffea $Z \rightarrow \mu\mu$)
 - GPU workers (NEMO / GridKa) in progress
 - Onboard more users and real HEP analyses

Backup

Legacy Implementation (on bms1)



Sign in

Username:

jmustafi

Password:

.....

Sign in



File Edit View Run Kernel Tabs Settings Help



+



/ my_jupyterhub /



Name

Modified

coffea

last mo.

jupy

last mo.

jupyterhub

8mo ago

nginx

last mo.

rocky

last mo.

Y: docker-compose...

last mo.

nextsteps.txt

34m ago

restart.sh

last mo.

Untitled.ipynb

now

Untitled.ipynb



Code



```
[*]: from dask_gateway import Gateway

gateway = Gateway("http://lx5.etp.kit.edu:8000", auth="jupyterhub")

cluster = gateway.new_cluster()
print(gateway.list_clusters())
```

[]:

[]:

[]:

○ (normalizing_flows) [jmustafi@lx5 my_jupyterhub]\$ docker exec -it ef073779a78d bash
[root@lx5 srv]# condor_q

-- Schedd: lx5.etp.kit.edu : <129.13.102.145:45699?... @ 09/27/25 12:51:36

OWNER	BATCH_NAME	SUBMITTED	DONE	RUN	IDLE	TOTAL	JOB_IDS
jmustafi	ID: 218	9/27 12:48	_	1	_	1	218.0

Total for query: 1 jobs; 0 completed, 0 removed, 0 idle, 1 running, 0 held, 0 suspended

Total for all users: 1 jobs; 0 completed, 0 removed, 0 idle, 1 running, 0 held, 0 suspended

[root@lx5 srv]# █

```
[1]: from dask_gateway import Gateway

gateway = Gateway("http://lx5.etp.kit.edu:8000", auth="jupyterhub")

cluster = gateway.new_cluster()
print(gateway.list_clusters())

[ClusterReport<name=03c499d4891b49c3a52ad0398db40fe6, status=RUNNING>]
```

```
[1]: from dask_gateway import Gateway

gateway = Gateway("http://lx5.etp.kit.edu:8000", auth="jupyterhub")

cluster = gateway.new_cluster()
print(gateway.list_clusters())

[ClusterReport<name=03c499d4891b49c3a52ad0398db40fe6, status=RUNNING>]
```

```
[2]: cluster.scale(2)
```


-- Schedd: lx5.etp.kit.edu : <129.13.102.145:45699?... @ 09/27/25 12:55:29

OWNER	BATCH_NAME	SUBMITTED	DONE	RUN	IDLE	TOTAL	JOB_IDS
jmustafi	ID: 218	9/27 12:48	—	1	—	1	218.0
jmustafi	ID: 219	9/27 12:54	—	—	1	1	219.0
jmustafi	ID: 220	9/27 12:54	—	—	1	1	220.0

Total for query: 3 jobs; 0 completed, 0 removed, 2 idle, 1 running, 0 held, 0 suspended

Total for all users: 3 jobs; 0 completed, 0 removed, 2 idle, 1 running, 0 held, 0 suspended

[root@lx5 srv]# █

```
[1]: from dask_gateway import Gateway

gateway = Gateway("http://lx5.etp.kit.edu:8000", auth="jupyterhub")

cluster = gateway.new_cluster()
print(gateway.list_clusters())

[ClusterReport<name=03c499d4891b49c3a52ad0398db40fe6, status=RUNNING>]
```

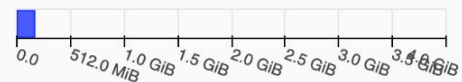
```
[ ]: cluster.scale(2)
```

```
[3]: print("Dashboard:", cluster.dashboard_link)
```

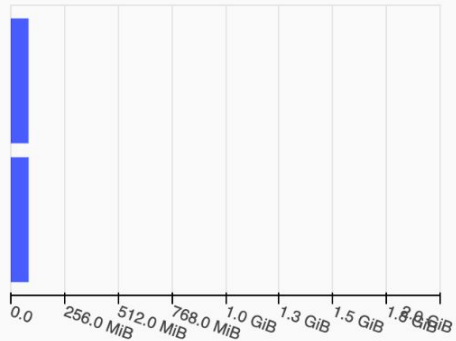
Dashboard: <http://lx5.etp.kit.edu:8000/clusters/03c499d4891b49c3a52ad0398db40fe6/status>

[Status](#)[Workers](#)[Tasks](#)[System](#)[Profile](#)[Graph](#)[Groups](#)[Info](#)[More...](#)[Documentation](#)

Bytes stored: 170.54 MiB



Bytes stored per worker

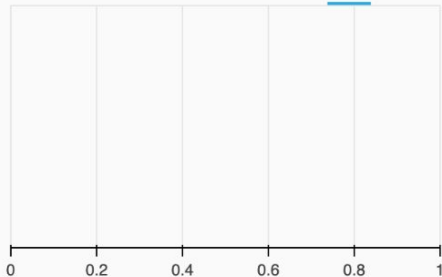


Processing

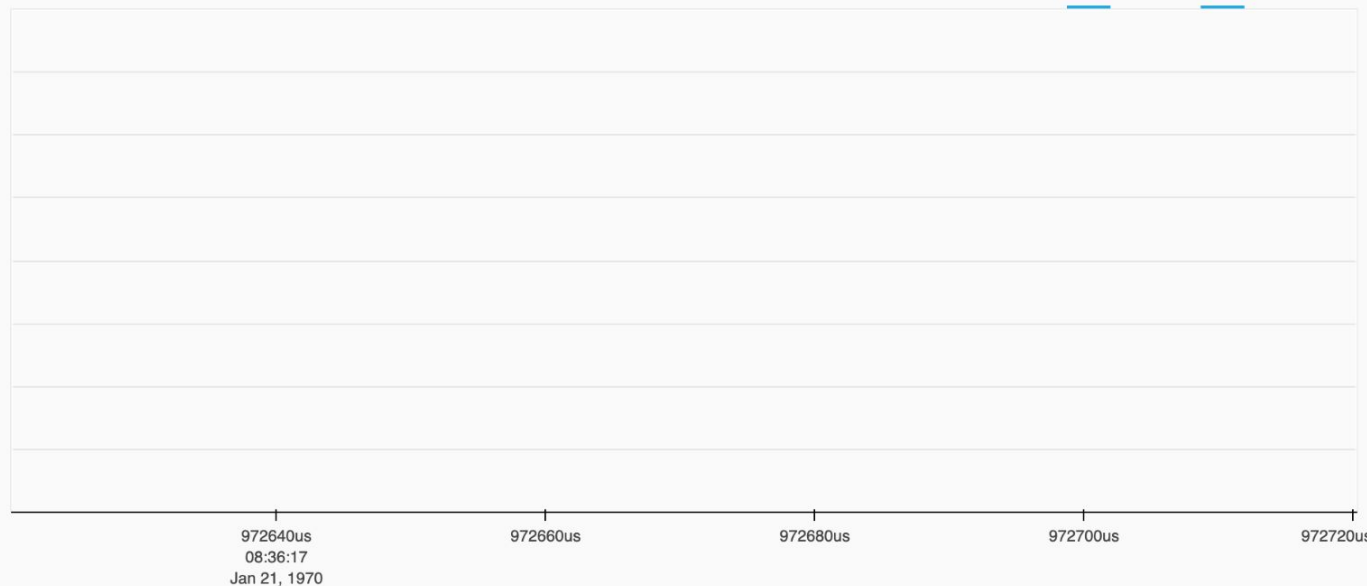
CPU

Data Transfer

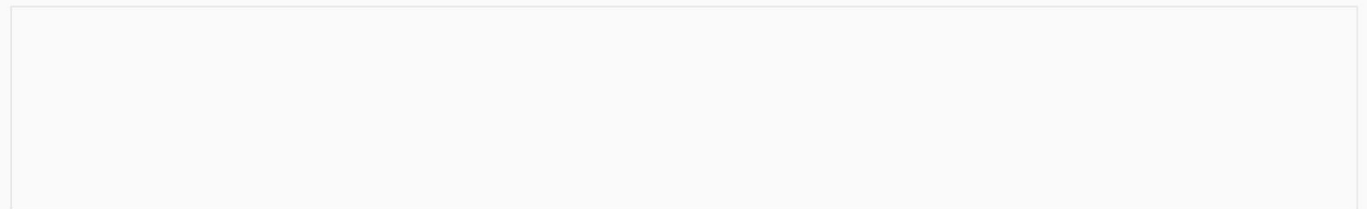
Tasks Processing



Task Stream



Progress



```
[6]: print("Dashboard:", cluster.dashboard_link)
```

Dashboard: <http://lx5.etp.kit.edu:8000/clusters/03c499d4891b49c3a52ad0398db40fe6/status>

```
[14]: from dask.distributed import Client
```

```
client = cluster.get_client()
```

```
[15]: import dask.array as da
```

```
x = da.random.random((10_000, 10_000), chunks=(2_000, 2_000))
```

```
y = da.random.random((10_000, 10_000), chunks=(2_000, 2_000))
```

```
# Matrix multiplication
```

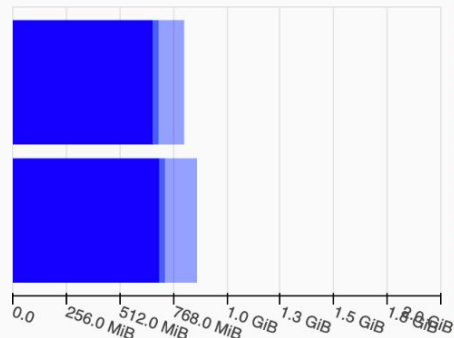
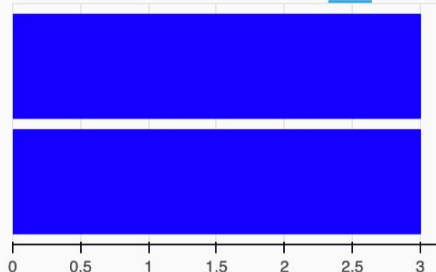
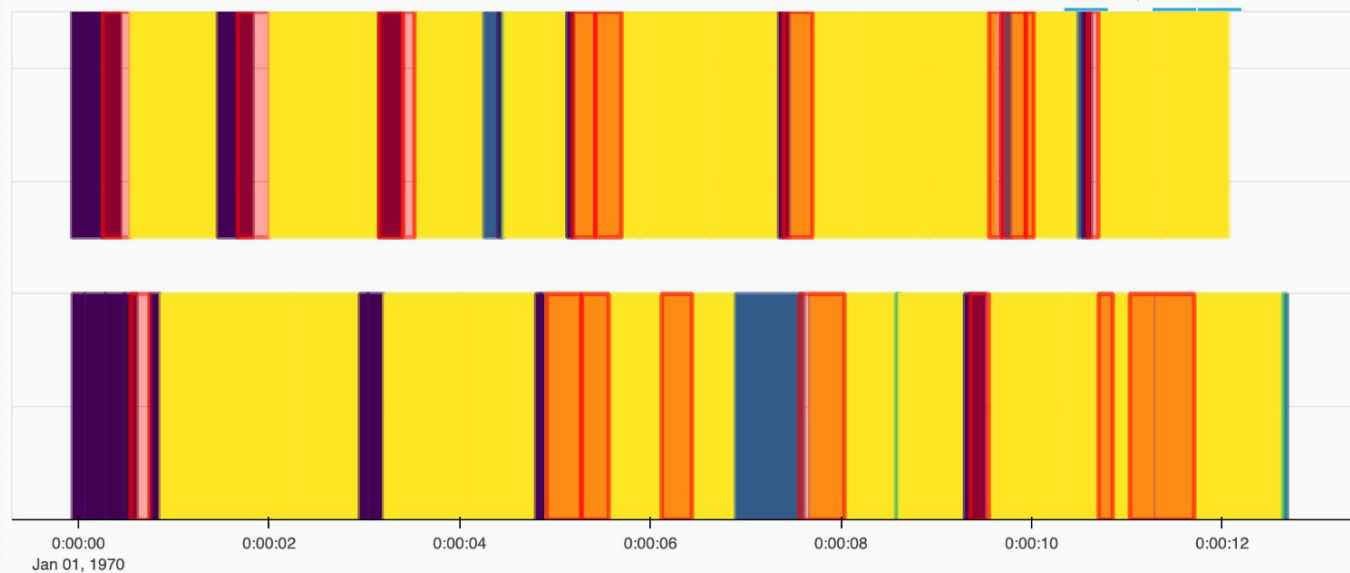
```
z = x @ y
```

```
# Trigger computation
```

```
result = z.mean().compute()
```

```
print(result)
```

2499.89528437347

[Status](#)[Workers](#)[Tasks](#)[System](#)[Profile](#)[Graph](#)[Groups](#)[Info](#)[More...](#)[Documentation](#)**Bytes stored: 1.66 GiB****Bytes stored per worker****Processing** CPU Data Transfer**Tasks Processing****Task Stream****Progress -- total: 288, waiting: 197, queued: 26, processing: 5, in-memory: 32, no-worker: 0, erred: 0**

chunk_sum	24 / 125
chunk_sum-pa...	8 / 50
random_sample	23 / 50
chunk_sum-ag...	3 / 25
mean_chunk	2 / 24
mean_combine...	0 / 12
finalize-hlg...	0 / 1
mean_agg-agg...	0 / 1